

Week2 task

Question 1

Correct

Mark 1.00 out of 1.00

Flag question

Question text

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s.

Aim is to find the number of 0s. Write a program using Divide and

Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m - Size of array

Next m lines Contains m numbers - Elements of an array

Output Format

First Line Contains Integer - Number of zeroes present in the given array.

Answer:(penalty regime: 0 %)

Ace editor not ready. Perhaps reload page?

Falling back to raw text area.

Sol:

```
#include <stdio.h>
```

```
int findFirstZero(int arr[], int low, int high)
```

```
{
```

```
    if (low > high)
```

```
        return -1;
```

```
    int mid = (low + high) / 2;
```

```
    if (arr[mid] == 0 && (mid == 0 || arr[mid - 1] == 1))
```

```
        return mid;
```

```
    if (arr[mid] == 1)
```

```
        return findFirstZero(arr, mid + 1, high);
```

```
    else
```

```
        return findFirstZero(arr, low, mid - 1);
```

```

}

int main()
{
    int m, arr[100];

    scanf("%d", &m);

    for (int i = 0; i < m; i++)
    {
        scanf("%d", &arr[i]);
    }

    int firstZero = findFirstZero(arr, 0, m - 1);

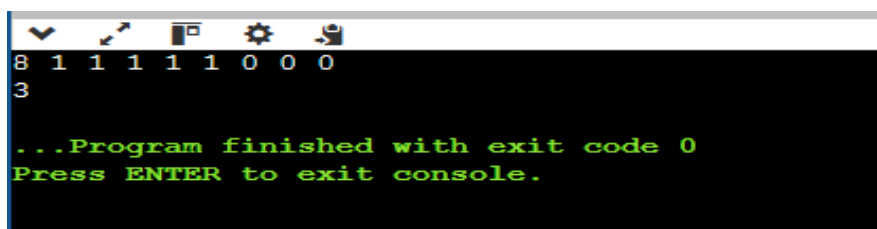
    if (firstZero == -1)
        printf("0");

    else
        printf("%d", m - firstZero);

    return 0;
}

```

Sample output:



```

8 1 1 1 1 1 0 0 0
3

...Program finished with exit code 0
Press ENTER to exit console.

```

QUESTION 2:

Given an array `nums` of size `n`, return the majority element.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

n == nums.length

1 <= n <= 5 * 10⁴

-231 <= nums[i] <= 231 - 1

For example:

Input	Result
-------	--------

3	
---	--

3 2 3	
-------	--

3	
---	--

7	
---	--

2 2 1 1 1 2 2	
---------------	--

2	
---	--

Answer: (penalty regime: 0 %)

Sol:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, nums[50000];
```

```
    int candidate = 0, count = 0;
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &nums[i]);
```

```
        if (count == 0)
```

```
        {
```

```

        candidate = nums[i];
    }

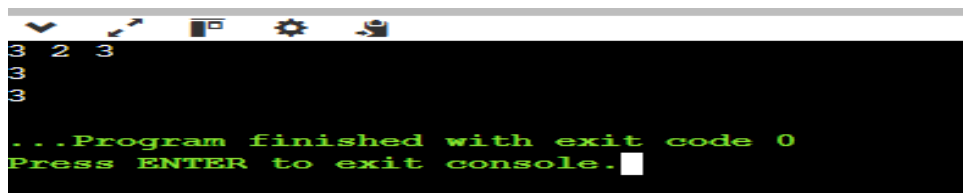
    if (nums[i] == candidate)
    {
        count++;
    }
    else
    {
        count--;
    }
}

printf("%d", candidate);

return 0;
}

```

Sample output:



```

3 2 3
3
3
...Program finished with exit code 0
Press ENTER to exit console.

```

QUESTION 3:

Problem Statement:

Given a sorted array and a value x , the floor of x is the largest element in array smaller than or equal to x . Write divide and conquer algorithm to find floor of x .

Input Format

First Line Contains Integer n - Size of array
 Next n lines Contains n numbers - Elements of an array
 Last Line Contains Integer x - Value for x

Output Format

First Line Contains Integer - Floor value for x

Answer:(penalty regime: 0 %)

Sol:

```
#include <stdio.h>
```

```
int findFloor(int arr[], int low, int high, int x)
```

```
{
```

```
    int floor = -1;
```

```
    while (low <= high)
```

```
    {
```

```
        int mid = (low + high) / 2;
```

```
        if (arr[mid] == x)
```

```
        {
```

```
            return arr[mid];
```

```
        }
```

```
        if (arr[mid] < x)
```

```
        {
```

```
            floor = arr[mid];
```

```
            low = mid + 1;
```

```
        }
```

```
    else
```

```
    {
```

```
        high = mid - 1;
```

```
    }
```

```
}
```

```
return floor;
```

```

}

int main()
{
    int n, arr[100], x;

    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

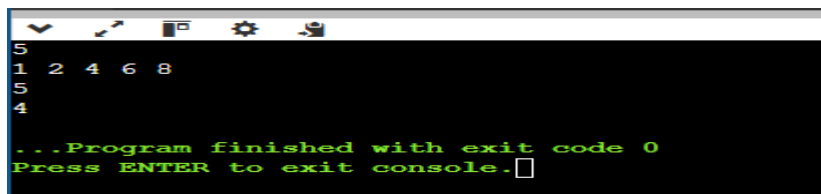
    scanf("%d", &x);

    printf("%d", findFloor(arr, 0, n - 1, x));

    return 0;
}

```

Sample output:



```

5
1 2 4 6 8
5
4
...Program finished with exit code 0
Press ENTER to exit console.

```

QUESTION 4:

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n - Size of array

Next n lines Contains n numbers - Elements of an array

Last Line Contains Integer x - Sum Value

Output Format

First Line Contains Integer - Element1

Second Line Contains Integer - Element2 (Element 1 and Elements 2 together sums to value "x")

Answer:(penalty regime: 0 %)

Sol:

```
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int key)
{
    if (low > high)
        return 0;

    int mid = (low + high) / 2;

    if (arr[mid] == key)
        return 1;

    if (arr[mid] < key)
        return binarySearch(arr, mid + 1, high, key);

    else
        return binarySearch(arr, low, mid - 1, key);
}

int main()
{
    int n, arr[100], x;

    int found = 0;

    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);
```

```

for (int i = 0; i < n; i++)
{
    int required = x - arr[i];
    if (binarySearch(arr, i + 1, n - 1, required))
    {
        printf("%d\n", arr[i]);
        printf("%d", required);

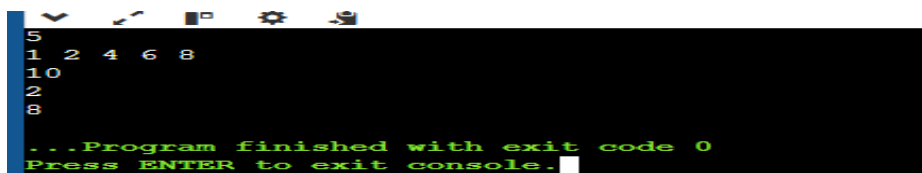
        found = 1;
        break;
    }
}

if (found == 0)
{
    printf("No");
}

return 0;
}

```

Sample output:



```

5
1 2 4 6 8
10
2
8
...Program finished with exit code 0
Press ENTER to exit console.

```

QUESTION 5:

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5	
67 34 12 98 78	
12 34 67 78 98	

Sol:

```
#include <stdio.h>

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++)
        {
            if (arr[j] < pivot)
            {
                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }

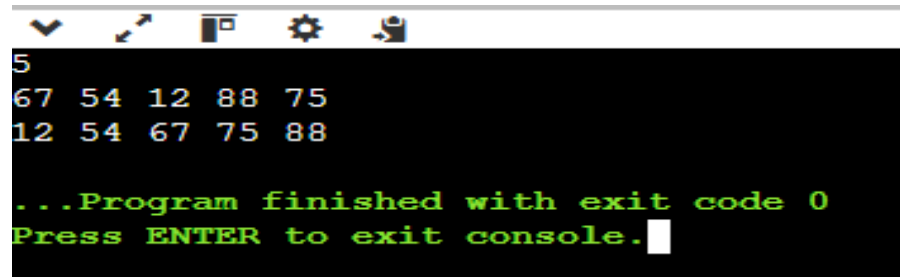
            int temp = arr[i + 1];
            arr[i + 1] = arr[high];
            arr[high] = temp;

            int pivotIndex = i + 1;
            quickSort(arr, low, pivotIndex - 1);
            quickSort(arr, pivotIndex + 1, high);
        }
    }
}

int main()
{
    int n, arr[100];
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

```
    return 0;  
}
```

Sample output:

A screenshot of a console window with a dark background. The window has a title bar with standard icons (checkmark, arrows, window, gear, and a person). The output text is as follows:

```
5  
67 54 12 88 75  
12 54 67 75 88  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

A white cursor is visible at the end of the last line.